

MTGNP v1.0 — Magic: The Gathering Network Protocol

CSNETWK Machine Problem — implementation of RFC 0001 (MTGNP v1.0). Python 3.10+, standard library only; no third-party packages.

Project Structure

```
project/
|-- server/                                Game Server package -- run: python3 -m server.main
|   |-- main.py                            entry point, socket accept loop, lifecycle (Dev 1)
|   |-- transport.py                       framing, PDU codec, ClientConn, dispatch table (Dev 1)
|   |-- heartbeat.py                       PING/PONG (Dev 1)
|   |-- lobby.py                           PLAYER_READY handling, LOBBY/SETUP/MULLIGAN (Dev 2)
|   |-- phase_engine.py                    phase/step machine, PHASE_TRANSITION, land drops (Dev 2)
|   |-- combat.py                          attackers/blockers/damage order/first strike (Dev 4)
|   |-- card_catalog.py                    loads shared JSON catalog; ILLEGAL_DECK validation (Dev 3)
|   |-- game_state.py                      authoritative data model; GameOver/ClientGone (Dev 3)
|   |-- state_view.py                      personalized GAME_STATE_UPDATE views, hidden hands (Dev 3)
|   |-- priority.py                        PRIORITY_GRANT/PASS, seq_num tokens, STALE_ACTION (Dev 3)
|   |-- stack.py                           LIFO stack, STACK_PUSH/RESOLVE, SBAs, triggers (Dev 3)
|   |-- effects.py                         card effects, mana payment, INSUFFICIENT_MANA (Dev 3)
|   |-- tests/
|       |-- test_card_catalog.py           unit: catalog + deck validation
|       |-- test_game_state.py             unit: data model + hidden info
|       |-- test_priority_stack.py         unit: tokens, LIFO stack, counterspell
|       |-- test_effects.py               unit: mana, effects, state-based actions
|       |-- test_integration_game.py       end-to-end protocol game (10 assertions)
|       |-- test_integration_combat.py     combat/effect mechanics ( 3 assertions)
|       |-- test_integration_edge.py       edge cases + TRIGGER_ORDER (18 assertions)
|-- shared/
|   |-- cards.json                         shared card catalog (out-of-band card data, RFC Sec. 1)
|-- client/
|   |-- main.py                           Player Client & rendering -- run: python3 -m client.main (Dev 4)
|-- decks/                                sample deck lists (one card instance ID per line)
|-- requirements.txt                       states that only the Python stdlib is required
|-- README.md                             this document (Markdown source)
|-- README.pdf                             this document (submitted deliverable)
```

The server is a Python package: one module per protocol concern, composed into the Server class in `server/main.py` via mixins. Method implementations are identical to the reference single-file build; only the file layout differs.

Deck files list one card instance ID per line; lines starting with `#` are comments. Card instance IDs are `<base>_<nnn>`, where the base is a key in `cards.json`.

Build and Run Instructions — Step by Step

Prerequisites: Python 3.10 or newer. No build step and no package installation are required — the project uses only the Python standard library.

Windows note: the command is `python` (or `py`), not `python3` — substitute it in every command below, e.g. `python server.py --verbose`. If Windows says 'Python was not found', install Python from python.org/downloads and tick 'Add python.exe to PATH' in the installer, then reopen the terminal. Verify with `python --version` (Linux/macOS: `python3 --version`).

Step 1. Unzip the project and open a terminal in the project folder (the one containing the server / package).

Step 2. Start the server in Terminal 1. It listens on TCP port 4444 (RFC 5.1):

```
python3 -m server.main --verbose
```

You should see [server] listening on port 4444. If the port is taken, add --port <n> here and on both clients.

Step 3. Start the first client in Terminal 2 (add --host <server-ip> if the server runs on another machine):

```
python3 -m client.main --id player_1 --deck decks/burn.txt --verbose
```

Step 4. Start the second client in Terminal 3. Any third connection attempt is refused by the server (RFC 5.1):

```
python3 -m client.main --id player_2 --deck decks/control.txt --verbose
```

Step 5. Type ready in each client and press Enter. When both players are ready the server deals opening hands and the mulligan phase begins.

Step 6. Mulligan: type keep to keep your hand or mull to redraw seven. After N mulligans, keep with keep <card_1> ... <card_N> to put N cards on the bottom of your library (London mulligan).

Step 7. Play the game. Whenever the client prints >>> You have priority you may act:

play mountain_003	play a land (own main phase, one per turn)
cast lightning_bolt_001 player_2	cast a spell; mana is paid automatically
pass	pass priority
attack goblin_guide_001	declare attackers (DECLARE_ATTACKERS step)
block wall_of_stone_001:goblin_guide_001	declare blockers (DECLARE_BLOCKERS step)
order <attacker> <blocker1> <blocker2>	damage order for a multi-blocked attacker
discard <card>	discard down to 7 at cleanup
yes / no	answer an optional trigger
hand / state	re-print your hand / full visible state
concede	concede the game
help	full command reference

Step 8. Game over: the winner and reason are announced. Both clients stay connected; type ready again to start a new game on the same connections (RFC 6.6).

Enabling Verbose Mode (rubric prerequisite)

Pass --verbose (or -v) to either program. Every PDU sent and received is then printed with a timestamp, direction label, peer, and the full JSON payload:

```
[14:03:22] SEND player_1 | {"type": "PRIORITY_GRANT", "player_id": "player_1", ...}
[14:03:23] RECV player_1 | {"type": "CAST_SPELL", "seq_num": 12, ...}
```

Without the flag both programs run quietly. Server debug flags: --time-limit <ms> (priority deadline, default 60000), --seed <n> and --first {0,1} (reproducible demos), --port <n>.

Tests

```
python3 server/tests/test_card_catalog.py # unit tests (catalog/decks)
python3 server/tests/test_game_state.py   # unit tests (data model)
python3 server/tests/test_priority_stack.py # unit tests (tokens/stack)
```

```
python3 server/tests/test_effects.py          # unit tests (effects/mana/SBAs)
python3 server/tests/test_integration_game.py  # 10 protocol assertions
python3 server/tests/test_integration_combat.py # 3 combat assertions
python3 server/tests/test_integration_edge.py  # 18 edge-case assertions
```

All suites pass (26 unit tests + 31 integration assertions), run from the project root.

test_integration_edge.py derives a deterministic RNG seed by replicating the server's seeded shuffle locally, so its scripted 10-turn game is fully reproducible.

Design Summary

Transport (RFC 5): every PDU is a 4-byte big-endian length prefix + UTF-8 JSON, max 65,535 bytes; the receiver reads exactly the framed bytes before parsing, and malformed payloads yield ERROR INVALID_JSON.

Architecture: the server runs one reader thread per client feeding a single event queue; the main thread runs the RFC Section 6 lifecycle (LOBBY, GAME_SETUP, MULLIGAN, IN_GAME, GAME_OVER, back to LOBBY on the same TCP connections). PING is answered with PONG in the reader thread (independent heartbeat counter). The client runs reader, heartbeat (PING every 30 s, 10 s PONG deadline) and stdin threads; it never simulates rules and re-renders each personalized GAME_STATE_UPDATE (own hand visible, opponent hand as a count only).

seq_num discipline (RFC 5.4): one monotonic server counter; a broadcast consumes one number. Clients echo the latest PRIORITY_GRANT for priority actions, the corresponding request PDU for MULLIGAN_CHOICE / DISCARD (a GAME_STATE_UPDATE) and combat declarations (the PHASE_TRANSITION), and any last server seq for CONCEDE. Mismatches get ERROR STALE_ACTION plus a re-granted token; after an illegal (non-stale) action the server re-issues PRIORITY_GRANT with the same seq_num (RFC Sec. 11 item 3).

Rules implemented: London mulligan; full phase sequence with first-turn draw skip; one land per turn at sorcery speed; implicit mana (declared mana_payment validated against untapped lands, which are tapped); LIFO stack with priority windows and caster-retains-priority; fizzle on illegal targets at resolution; state-based actions after every event (lethal damage, zero toughness, life at or below zero, active player loses simultaneous-death ties); summoning sickness; combat with attack tapping, one-attacker-per-blocker, multi-block damage ordering (ASSIGN_DAMAGE_ORDER, lethal-then-overflow, no trample), a first-strike step when relevant; cleanup discard-to-7 loop; wins by LIFE_ZERO, DECK_EMPTY, CONCEDE, DISCONNECT (including priority timeout); extra connections refused; lobby deck resubmission; duplicate ID rejection.

Card effects (more than the required 5): Lightning Bolt and Shock (damage), Counterspell (counters a stack item), Giant Growth (+3/+3 until end of turn), Healing Salve (life gain), Divination (draw 2), Gray Merchant of Asphodel (optional ETB drain-2 trigger exercised via TRIGGER_CHOICE / TRIGGER_CHOICE_RESPONSE), Festering Imp (death trigger; two dying simultaneously exercises TRIGGER_ORDER / TRIGGER_ORDER_RESPONSE), Goblin Guide (haste), Youthful Knight (first strike), vanilla creatures, five basic lands.

Known Limitations / Deviations from the RFC

1. A countered spell is announced with STACK_RESOLVE result FIZZLE; the RFC defines only RESOLVED | FIZZLE, with no dedicated 'countered' result.

2. Gray Merchant's drain is modelled as an optional ('you may') trigger so the TRIGGER_CHOICE flow of RFC 8.6.3 is exercised; declining discards the trigger.
3. ACTIVATE_ABILITY is answered with ILLEGAL_ACTION: the fixed card set defines no activated non-mana abilities, and mana is implicit per RFC 7.5; consequently the interactive client has no command that sends it (the server-side rejection path is fully implemented).
4. Summoning sickness is cleared for the active player's creatures during their Cleanup (equivalent, for this card set, to 'continuously under your control since your most recent turn began').
5. --seed and --first are non-RFC debug conveniences used by the tests and available for demos.

Work Distribution Matrix

[FILL IN before submission — replace names, tasks and percentages for your group per the Groupings section.]

Member	Tasks	%
(Name 1)	(e.g. framing, lobby, mulligan)	()
(Name 2)	(e.g. turn engine, priority and stack)	()
(Name 3)	(e.g. combat, SBAs, triggers)	()
(Name 4)	(e.g. client, tests, README)	()

AI Usage

The following AI tool was used in this project; per the course policy, usage is disclosed in full and every member is prepared to explain all of the code at the demo.

Tool: Claude (Anthropic), via the `claude.ai` chat interface.

What it was used for: Claude generated the initial complete implementation of `server.py`, `client.py`, `common.py`, `cards.json`, the sample decks, and all three test suites from the RFC document, and iteratively debugged them (notably: fixing STALE_ACTION token re-validation after a re-grant, and adding the same-seq_num PRIORITY_GRANT re-issue required by RFC Section 11 item 3). Claude also drafted this README.

What the group did: [FILL IN truthfully — e.g. reviewed the code against the RFC, ran and extended the test suites, made the following manual modifications: Do not overstate: the rubric treats inability to explain the code, or an inaccurate disclosure, as academic dishonesty.]